

2.3.2 解码调整

- 在生成器处理过程中添加额外控制，如调整超参数以实现更大的多样性，限制输出词汇等。

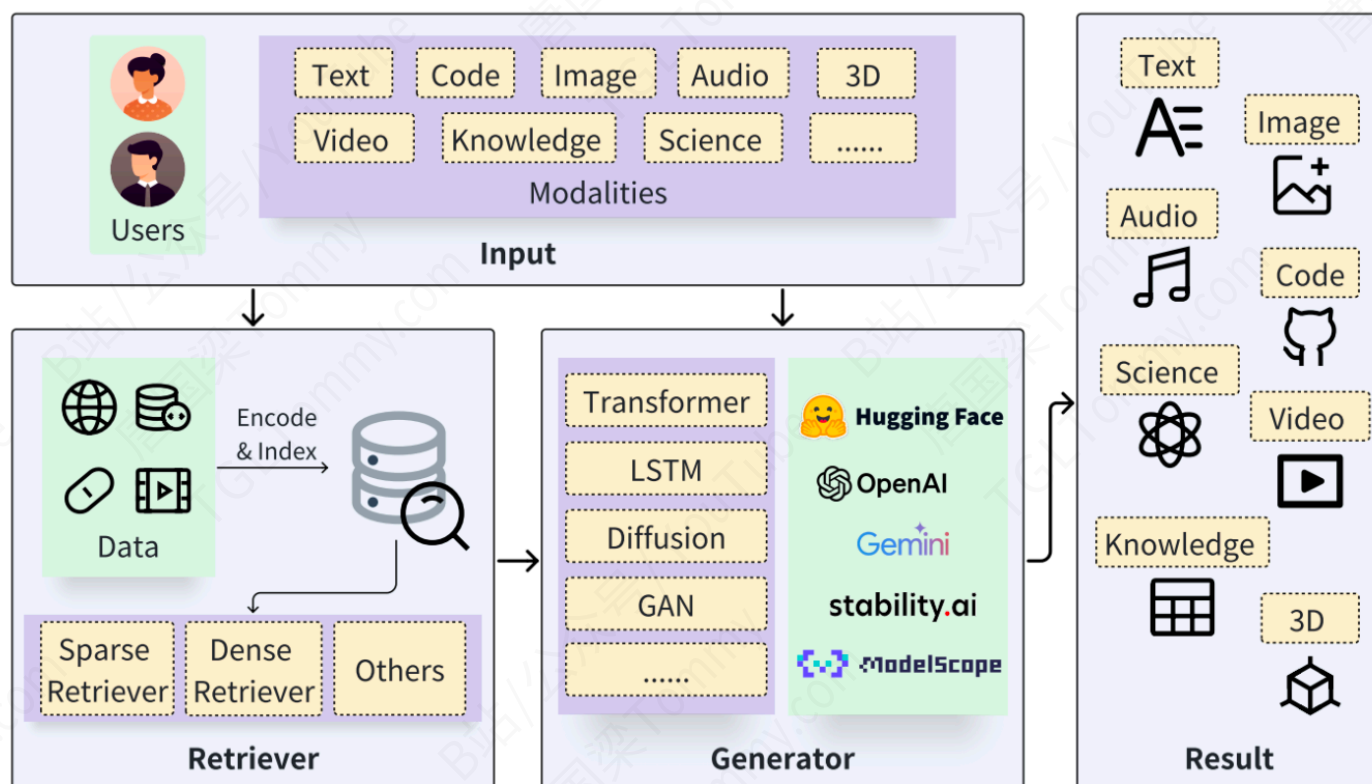
2.3.3 微调生成器

- 通过微调生成器来增强模型的领域知识或与检索器更好的匹配度。

2.4 高级RAG的特点

- 性能优化：**通过高级的索引方法和精细化的检索策略，高级RAG能够更快速、更准确地定位到查询相关的信息，提高了检索效率和生成内容的质量。
- 适应性强：**结合多种检索技术和策略，高级RAG能够根据不同的查询需求和信息类型，灵活调整检索策略，保证了结果的相关性和丰富性。
- 动态嵌入应用：**通过对嵌入模型的微调和动态调整，高级RAG在处理上下文变化时表现出更高的适应性和准确性，特别是在面对多样化和复杂查询时。
- 高级后处理：**采用先进的后处理技术，如提示压缩和智能重排序，进一步优化生成内容的准确性和一致性，显著提升用户体验。

五、RAG的核心组件



1. 知识库构建

构建高效、可靠的知识库是实现RAG系统的基础。以下是构建知识库的关键步骤及其组件的详细说明：

1.1 文档预处理

文档解析是知识库构建的首要步骤，旨在将各种格式的文档转换为机器可读的文本格式。

- **PyPDF2**：主要用于处理PDF文件，支持提取文本、分割和合并PDF页面。适用于简单的PDF文本提取任务，但可能在处理复杂布局时遇到限制。
- **PDF2Text**：一种更专注于将PDF文档转换为纯文本的工具，强调转换质量和速度，适合大规模的PDF文本提取需求。
- **GROBID**：利用机器学习方法处理PDF和其他类型文档的高级解析工具。能够识别文档的结构和元数据，适用于需要高精度文档结构化信息的场景。
- **OCR**：
 - 文字：Paddle-OCR, Rapid-OCR；
 - 表格：Camelot, Paddle-OCR, 阿里追光, Talbe Transformer；
 - 公式：Nougat, marker；

选择哪个工具取决于具体的需求：对于需要高精度结构化输出的应用，GROBID是更好的选择；而对于大量PDF文本提取，PDF2Text可能更为高效。

1. PyPDF2 : <https://github.com/py-pdf/pypdf.git>
2. GROBID : <https://github.com/kermitt2/grobid.git>
3. Paddle-OCR : <https://github.com/PaddlePaddle/PaddleOCR.git>
4. Rapid-OCR : <https://github.com/RapidAI/RapidOCR.git>

1.2 向量数据库选择

向量数据库用于存储文档的向量表示，支持高效的相似度搜索。

- **FAISS**：由Meta开发，专为高效相似度搜索和密集向量聚类设计。特别适用于处理极大规模的数据集。
- **Milvus**：一个开源的向量数据库，支持海量向量的存储和检索，提供了灵活的索引构建和搜索能力。适用于企业级应用。
- **ElasticSearch**：一个开源的搜索引擎，支持全文搜索及稠密向量的搜索。适合于文本搜索与简单向量搜索的场景。

选择向量数据库时，应考虑数据规模、检索速度和易用性。

1. FAISS : <https://github.com/facebookresearch/faiss.git>
2. Milvus : <https://github.com/milvus-io/milvus.git>
3. ElasticSearch : <https://github.com/elastic/elasticsearch.git>

1.3 Embedding模型选择

选择合适的嵌入模型对于生成高质量的文档表示至关重要：

- **BERT、RoBERTa、GPT**等预训练语言模型，能够捕获深层次的语义信息，适合于文本嵌入。
- **Sentence-BERT、SimCSE**等专为句子级别或段落级别的相似度计算优化的模型，提供了更加精细的文本向量化方式。

Embedding中文模型排名：(2024.04.02)

<https://huggingface.co/spaces/mteb/leaderboard>

Rank	Model	Model Size (GB)	Embedding Dimensions	Max Tokens	Average (35 datasets)	Classification Average (9 datasets)	Clustering Average (4 datasets)	Pair Classification Average (2 datasets)
1	acge_text_embedding	0.65	1024	1024	69.07	72.75	58.7	87.84
2	OpenSearch-text-hybrid		1792	512	68.71	71.74	53.75	88.1
3	stella-mrl-large-zh-v3.5-1792	1.3	1024	512	68.55	71.56	54.32	88.08
4	stella-large-zh-v3-1792d	1.3	1024	512	68.48	71.5	53.9	88.1
5	Baichuan-text-embedding		1024	512	68.34	72.84	56.88	82.32
6	stella-base-zh-v3-1792d	0.41	768	1024	67.96	71.12	53.3	87.93
7	Dmeta-embedding-zh	0.41	768	1024	67.51	70	50.96	88.92
8	xiaobu-embedding	1.3	1024	512	67.28	71.2	54.62	85.3
9	alime-embedding-large-zh	1.3	1024	512	67.17	71.35	54	84.34
10	gte-large-zh	0.65	1024	512	66.72	71.34	53.07	84.41

1. sentence-bert: <https://github.com/UKPLab/sentence-transformers.git>

2. SimCES: <https://github.com/princeton-nlp/SimCSE.git>

3. FlagEmbedding: <https://github.com/FlagOpen/FlagEmbedding.git>

4. BCEembedding: <https://github.com/netease-youdao/BCEembedding.git>

1.4 索引写入策略

有效的索引写入策略可以提高检索的准确性和效率。

- **文本预处理**：包括去除停用词、词干提取、小写化等，以减少噪声并提高向量表示的质量。
- **文本框大小调整**：根据实际需求调整文本块的大小，较小的块有助于提高检索的精确度，而较大的块可能更适合概览式的搜索。
- **文本块重叠处理**：通过让文本块之间有一定的重叠，可以避免重要信息被切分至不同块中而影响检索效果。

2. 检索器